

Chapter 12

Properties of Context-free Languages

12.1 Closure Properties

Let us study the various closure properties of CFLs.

1. Union

Let L_1 and L_2 be two CFLs accepted by the CFGs $G_1 = (V_1, \Sigma, P_1, S_1)$ and $G_2 = (V_2, \Sigma, P_2, S_2)$ respectively. Then $L_1 \cup L_2$ will be accepted by the CFG $G = (V, \Sigma, P, S)$ where

$$\begin{aligned} V &= V_1 \cup V_2 \cup \{S\}, \text{ and} \\ P &= P_1 \cup P_2 \cup \{S \longrightarrow S_1 \mid S_2\}. \end{aligned}$$

2. Concatenation

Let L_1 and L_2 be two CFLs accepted by the CFGs $G_1 = (V_1, \Sigma, P_1, S_1)$ and $G_2 = (V_2, \Sigma, P_2, S_2)$ respectively. Then $L_1 \cdot L_2$ will be accepted by the CFG $G = (V, \Sigma, P, S)$ where

$$\begin{aligned} V &= V_1 \cup V_2 \cup \{S\}, \text{ and} \\ P &= P_1 \cup P_2 \cup \{S \longrightarrow S_1 S_2\}. \end{aligned}$$

3. Star

Let L_1 be a CFL accepted by the CFG $G_1 = (V_1, \Sigma, P_1, S_1)$. Then L_1^* will be accepted by the CFG $G = (V, \Sigma, P, S)$ where

$$\begin{aligned} V &= V_1 \cup \{S\}, \text{ and} \\ P &= P_1 \cup \{S \longrightarrow S_1 S \mid \epsilon\}. \end{aligned}$$

4. Reversal

Let L_1 be a CFL accepted by the CFG $G_1 = (V, \Sigma, P, S)$. Then $\text{rev}(L_1)$ will be accepted by the CFG $G = (V, \Sigma, P_R, S)$ where for every rule $A \longrightarrow X_1 X_2 \dots X_k$ in P , add the rule

$$A \longrightarrow X_k \dots X_2 X_1$$

to P_R .

5. Homomorphism and Inverse Homomorphism

CFLs are also closed under homomorphism and inverse homomorphism. The construction is left as an exercise.

Exercise 24. Show that CFLs are closed under homomorphism and inverse homomorphism.

(*Hint:* For homomorphism start with a CFG and for inverse homomorphism start with a PDA.)

6. Intersection with a Regular language

Let L_1 be a CFL and L_2 be a regular language, then $L_1 \cap L_2$ is a CFL.

The idea is to take a PDA N for L_1 and a DFA M for L_2 and construct a PDA (say N') for $L_1 \cap L_2$. N' will be a “product automaton” of M and N , making a move according to both M and N at each step simultaneously. In addition, N' will use its own stack to simulate the stack of N .

Formally, let $N = (Q_N, \Sigma, \Gamma, \delta_N, q_{0_N}, F_N)$ and $M = (Q_M, \Sigma, \delta_M, q_{0_M}, F_M)$. We construct $N' = (Q, \Sigma, \Gamma, \delta, q_0, F)$ as follows:

- $Q = Q_N \times Q_M$
- $q_0 = (q_{0_N}, q_{0_M})$
- $F = F_N \times F_M$
- $((r, s), Y) \in \delta((p, q), a, X)$ if
 - $(r, Y) \in \delta_N(p, a, X)$, and
 - $s = \delta_M(q, a)$ if $a \in \Sigma$, and $s = q$ if $a = \epsilon$.

12.1.1 Non-closure under certain operations

What about other set operations such as intersection, complement and set difference?

It turns out that CFLs are *not* closed under these operations.

- Consider the two languages

$$\begin{aligned} L_1 &= \{a^n b^n c^m \mid n, m \geq 0\} \\ L_2 &= \{a^n b^m c^m \mid n, m \geq 0\} \end{aligned}$$

Here is a CFG for L_1 :

$$\begin{aligned} S &\longrightarrow S_1 C \\ S_1 &\longrightarrow a S_1 b \mid \epsilon \\ C &\longrightarrow c C \mid \epsilon \end{aligned}$$

Similarly one can construct a CFG for L_2 as well. But now we can write our favourite non context-free language $L = \{a^n b^n c^n \mid n \geq 0\}$ as,

$$L = L_1 \cap L_2.$$

Hence CFLs are not closed under intersection.

- If CFLs were closed under complement, then by DeMorgan's law they would be closed under intersection as well. Hence CFLs are not closed under complement.
- For a language $L \subseteq \Sigma^*$, $\bar{L} = \Sigma^* \setminus L$. This shows that CFLs are not closed under set difference as well.

12.1.2 Some Applications

1. Let

$$L_1 = \{w \in \{a, b, c\}^* \mid \#_a(w) = \#_b(w) = \#_c(w)\}.$$

Note that $L_1 \cap L(a^*b^*c^*) = \{a^n b^n c^n \mid n \geq 0\}$. Since $L(a^*b^*c^*)$ is a regular language and the language on the right hand side of the equal sign is not a CFL therefore L_1 is not a CFL.

2. Show that

$$L_2 = \{a^n b^n a^{2m} b^{2m} \mid n, m \geq 0\}$$

is context-free.

We use the fact that $L' = \{a^n b^n \mid n \geq 0\}$ is context-free. Consider the homomorphism h defined as

$$\begin{aligned} h(a) &= aa \\ h(b) &= bb. \end{aligned}$$

Then $h(L') = \{a^{2n} b^{2n} \mid n \geq 0\}$. Now observe that $L_2 = L' \cdot h(L')$. Therefore L_2 is context-free.

Chapter 13

Deterministic Context-free Languages

13.1 Deterministic Pushdown Automata

Definition 13.1.1. A *deterministic pushdown automaton* (in short DPDA), $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ is a PDA such that,

- for all $q \in Q$, $a \in \Sigma_\epsilon$ and $X \in \Gamma$, $\delta(q, a, X)$ has at most one element, and
- for all $q \in Q$ and $X \in \Gamma_\epsilon$, if $\delta(q, \epsilon, X) \neq \emptyset$ then $\delta(q, a, X) = \emptyset$ for all $a \in \Sigma$.

The class of languages accepted by DPDAs are known as *deterministic context-free languages* (or in short, DCFLs).

Exercise 25. Construct a DPDA for the language $L_1 = \{0^n 1^n \mid n \geq 0\}$.

- Not all context-free languages have a DPDA. An example is the language PALINDROMES, consisting of all strings that are palindromes. Intuitively this is because the automaton inherently *requires* the use of non-determinism to guess the middle position of a string. The proof to show that PALINDROMES does not have a DPDA accepting it is beyond the scope of this course.
- Every DCFL will have an unambiguous grammar. Is the reverse true?
- Most programming languages can be described using DPDAs.

Exercise 26. Show that there is a CFL that is not a DCFL and has an unambiguous grammar.

13.2 Closure Properties of DCFLs

- Closed under complement
- Not closed under union.
- Not closed under intersection.

Chapter 14

Turing Machines

So far we have seen the following:

Finite Automaton: Finite control (set of states). Input is read in one direction. No memory.

Pushdown Automaton: Finite control (set of states). Input is read in one direction. Memory is restricted to be in the form of a stack (however the stack length is unbounded).

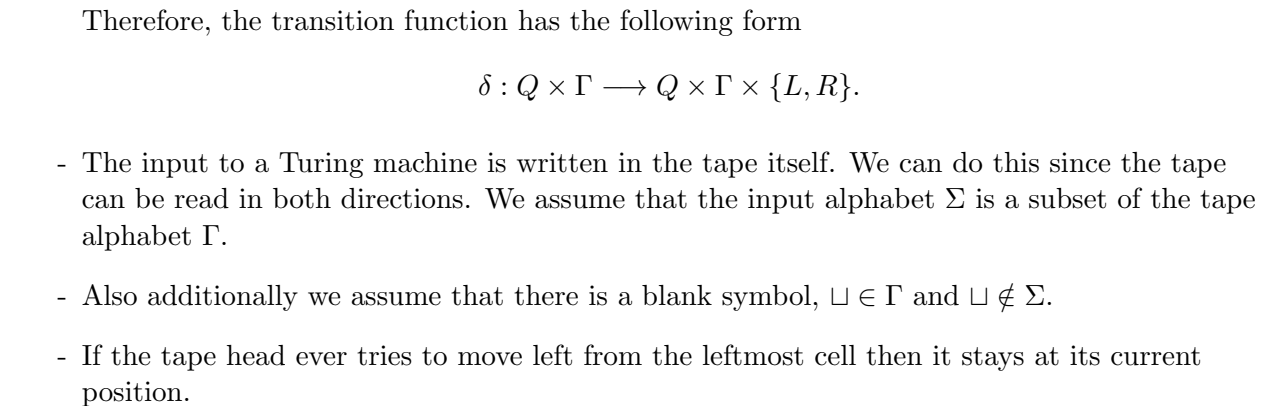
14.1 Turing Machine

14.1.1 The model

- Introduced by Alan Turing in 1936.
- Theoretical model. It can model the computation of any computing device.
- Finite control. Infinite memory in the form of a tape (all cells are accessible). Input can be read in both directions.
- Has a designated accept state and reject state. The computation halts when the Turing machine enters one of these two states.
- A transition rule of a Turing machine has the following form

$$\delta(p, X) = (q, Y, L).$$

This means that from state p , on reading the symbol X on the tape, the machine moves to state q , replaces X with Y and moves the tape head to the left. This is illustrated in the diagram below.

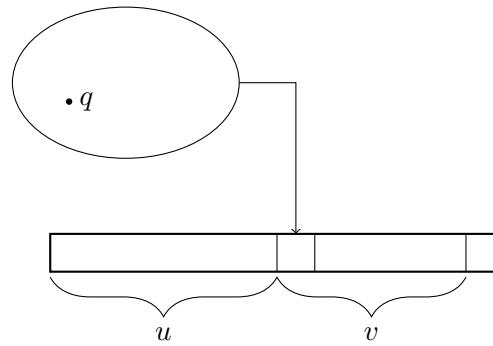


Definition 14.1.1. A Turing machine (denoted as TM in short) is the tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_A, q_R)$, where

- ### 14.1.2 Configuration of a TM

- A *configuration* of the TM M is a snapshot of the TM at a given instant. It consists of
 - i) the current state,
 - ii) the current tape contents, and

- iii) the current position of the tape head.
- We denote a configuration as uqv , where $u, v \in \Gamma^*$ and $q \in Q$, if
 - i) q is the current state,
 - ii) uv is the current tape contents, and
 - iii) the tape head points to the first symbol in v (as shown below).



The configuration uqv .

- A configuration C_1 *yields* a configuration C_2 in one step (denoted as $C_1 \Rightarrow C_2$), if M can go from C_1 to C_2 in one transition step.
- Some standard configurations
 - Start configuration*: q_0w , where w is the input to the Turing machine. That is the head is pointing to the leftmost tape symbol.
 - Accept configuration*: Any configuration of the form uq_Av , where $u, v \in \Gamma^*$.
 - Reject configuration*: Any configuration of the form uq_Rv , where $u, v \in \Gamma^*$.
- M *accepts* a string w , if $\exists$ a sequence of configurations $C_0, C_1, \dots, C_k$, such that,
 - i) C_0 is the start configuration,
 - ii) for $1 \leq i \leq k$, $C_{i-1} \Rightarrow C_i$, and
 - iii) C_k is an accept configuration.
- M *rejects* a string w , if $\exists$ a sequence of configurations $C_0, C_1, \dots, C_k$, such that,
 - i) C_0 is the start configuration,
 - ii) for $1 \leq i \leq k$, $C_{i-1} \Rightarrow C_i$, and
 - iii) C_k is a reject configuration.
- Observe that there can be strings on which M neither accepts nor rejects (basically goes into an infinite loop).
- M is said to be a *halting* TM, if $\forall w \in \Sigma^*$, M either accepts w or rejects w .
- $L(M) = \{w \mid M \text{ accepts } w\}$.

- A language L is said to be *Turing-recognizable* if there exists a TM M such that $L = L(M)$.
- A language L is said to be *Turing-decidable* (or simply *decidable*) if there exists a halting TM M such that $L = L(M)$.

Remark. Turing-recognizable languages are also called *recursively enumerable* languages and Turing-decidable languages are also called *recursive* languages.

14.1.3 An Example

Consider the non-CFL

$$L = \{a^n b^n c^n \mid n \geq 0\}.$$

We give the intuition behind the construction of a TM for L .

1. First make a pass through the input and check whether it is of the form $a^*b^*c^*$ or not. If not then enter reject state else return to the leftmost cell.
2. Do the following until all occurrences of one of the three symbols are completely crossed out.
 - (i) Cross the leftmost uncrossed a .
 - (ii) Move to the leftmost uncrossed b and cross it.
 - (iii) Move to the leftmost uncrossed c and cross it.
 - (iv) Move back to the leftmost uncrossed a .
3. Make one more pass through the tape. If all symbols are crossed out then accept else reject.

Exercise 27. Give the formal definition of a TM for the above language.

Note 8. As you can see even for a simple language, designing a TM can be a cumbersome experience. We will usually give a high level description for a decidable or Turing-recognizable language. The important point is that *anything that can be computed can be computed using a TM*.

14.1.4 Variants of the TM

Let us consider some variants of the TM and argue about their computational power.

1. **Multitape TM:** The TM is allowed to have k tapes. It has a separate head in each of the k tapes. In each transition step, each tape replaces the bit under its respective head and moves left/right.

We can simulate all the k tapes using a single tape. The contents of the different tapes are written side by side, separated by some marker symbol. Two issues arise in this case.

- What if the contents of a tape increases?
For each additional symbol added to a tape we shift the contents on the right side, to the right by one cell.
- How do we keep track of the head positions for each individual tape?
For each tape symbol, say a , in the original TM we add another symbol $\bar{a}$. This new symbol is used to keep track of the head position in each tape's portion.

2. **Two stacks:** The two stacks store the contents of the tape on the left side and right side of the tape head respectively.
3. **Queue:** Can be simulated using two stacks.

Exercise 28. Read Chapters 3.1, 3.2.

14.2 Nondeterministic Turing Machine

Given a TM M and an input w to M , from any configuration C , M can only move to a unique configuration C' . However we can also define a nondeterministic variant in which the machine can move to multiple configurations simultaneously.

- **Transition function:** A nondeterministic Turing machine is defined in the same way as a deterministic TM, except for the transition function. The transition function of a nondeterministic Turing machine is defined as

$$\delta : Q \times \Gamma \longrightarrow 2^{Q \times \Gamma \times \{L,R\}}.$$

In other words, from a single (*state, symbol*) pair the machine can have multiple transitions.

- **Configuration:** Given a nondeterministic TM M and an input w to M , from any configuration C , M can move to multiple configurations in one step.
- **Acceptance criterion:** The machine accepts if *some* sequence of transitions leads to an accept state.

14.2.1 Equivalence of Deterministic and Nondeterministic TMs

Clearly a deterministic TM is also a nondeterministic TM.
For the other direction we simulate a

14.2.2 Examples

Checking Non-Primality

Consider the language

$$L_C = \{0^p \mid p \text{ is a composite}\}.$$

We will design a nondeterministic TM for L_C as follows.

Input: A string w

1. Nondeterministically write l_1 a 's and l_2 b 's on the tape, where l_1 and l_2 are two numbers such that $2 \leq l_1, l_2 < p$.
2. Repeat the following until all a 's are crossed off.
 - i. Alternately cross off all b 's and equal number of 0's in the string w .
 - ii. If no 0's remain then reject.
 - iii. Uncross all b 's.
 - iv. Cross off one a .
3. Accept if all 0's in w are crossed off, else reject.

Subset Sum

Given a set of integers S and a number k , is there a subset $S' \subseteq S$ such that the sum of the elements in S' is k ?

We can formally state the above problem in a language form.

$$L_{SS} = \{\langle S, k \rangle \mid \exists S' \subseteq S \text{ and } \sum_{a \in S'} a = k\}$$

where $\langle S, k \rangle$ is an encoding of the set S and the number k .

The algorithm is simple. We nondeterministically guess a subset S' of S and compute its sum. If it equals k then accept else reject.

14.2.3 Configuration Graph

The concept of configuration graph is very important to the study of languages.

Definition 14.2.1. Given a Turing machine M (may be deterministic or nondeterministic) and an input x to M , the *configuration graph* of M on x , denoted as $G_{M,x}$ is defined as follows:

- the vertices of $G_{M,x}$ are the configurations of the machine M on input x , and
- there is an edge from a configuration C_1 to a configuration C_2 if there is a one step transition of M from C_1 to C_2 , on the input x .

A *computation path* is a path in $G_{M,x}$ starting from the start configuration.

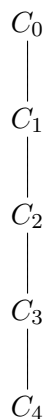
From the definition we can immediately observe the following Proposition.

Proposition 20. M accepts x if and only if there is a path from the start configuration to an accept configuration in $G_{M,x}$.

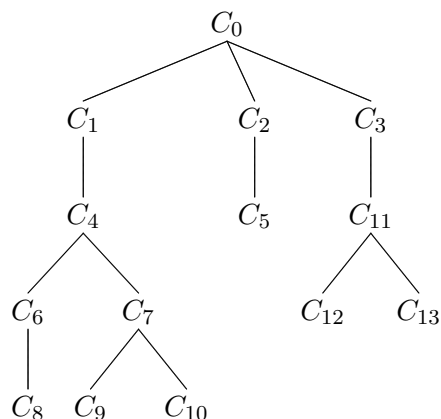
We will now discuss some properties regarding the configuration graph. Let M be a TM and x be an input to M .

- The definition of configuration graph only makes sense if both a machine and an input is provided.
- If M is deterministic then the outdegree of $G_{M,x}$ is at most one. On the other hand, if M is nondeterministic then the outdegree can be arbitrary.
- A vertex in $G_{M,x}$ has outdegree zero if and only if it corresponds to an accept or reject configuration.
- Technically the configuration graph can be infinite. This is because a configuration depends on the tape contents which can be unbounded. However later when we move to complexity theory, we will be able to bound the space used on the tape, as a function of the input size. As a result, the configuration graph will also be finite in such cases.
- Although by definition $G_{M,x}$ contains all possible configurations, but we will usually be interested in only those configurations in $G_{M,x}$ that are reachable from the start configuration.

Examples



Configuration graph of a deterministic machine.



Configuration graph of a nondeterministic machine.

14.2.4 Equivalence of Deterministic and Nondeterministic TMs

Theorem 21. *The class of languages accepted by deterministic TMs is the same as the class of languages accepted by nondeterministic TMs.*

One direction is trivial since deterministic TMs are also nondeterministic TMs.

To see the other direction, consider a NDTM N . We will give the sketch of the design of a deterministic TM M , such that $L(M) = L(N)$. Let x be an input. The machine M essentially does a breadth first search on the configuration graph $G_{N,x}$ by traversing it in a level wise manner.

- i) M maintains a queue data structure which initially has a start configuration.
- ii) In every iteration, M dequeues a configuration and enqueues all its children into the queue.
- iii) If an accept configuration is encountered at any stage then M halts and accepts.
- iv) If the queue becomes empty (that is all configurations reachable from the start configuration are traversed) then M halts and rejects.

Note 9. We both BFS and DFS are graph traversal algorithms, we cannot do a DFS to traverse $G_{N,x}$. This is because even for an input which is in $L(N)$, N might have a computation path of infinite length. DFS will remain stuck on this path forever.

14.2.5 Definition of Algorithm

Hypothesis 22 (Church Turing Thesis). *Anything that can be computed can be computed using a Turing machine.*

The Church-Turing thesis essentially says that our notion of algorithms as being a step by step procedure of solving a problem, is the same as that of Turing machine algorithms.

Exercise 29. Read Chapter 3.3.